

Analysis: Impartial Offerings

One observation that helps us find a solution is: we will never need to give any pet more than N treats. So for each pet, we need to consider at most N options for treats, $1, 2, \dots, N$. We can quickly verify that this observation is true with a simple argument. If there is a positive integer X such that no pet receives X and some pets receive $X + 1$, then we can give all those pets X instead, keep everything valid and reduce the total amount of treats.

For Test Set 1, limits are small enough for a brute force solution. For each pet, we can try all the numbers $(1, 2, \dots, N)$ of treats, and check if that assignment fulfills our criteria. Then we output the minimum total from among the valid assignments. There are N^N possible assignments. For each assignment, we can check the validity criteria by checking that all pairs of pets satisfy the impartiality rule in $O(N^2)$ time. This makes the overall time complexity $O(N^{N+2})$, which is sufficient for Test Set 1.

For Test Set 2, we need to find the solution faster.

All pets of the same size must get the same amount of treats, and any pet must get strictly more treats than all pets strictly smaller than it. That means that if we sort the pets by size (in non-decreasing order), the number of treats each pet must get is also sorted. Since we need to minimize the number of treats, we can start by giving 1 treat to the smallest animal. Then, for every animal in the sorted list, we can give it:

- the same number of treats as the previous one, if it is the same size as the previous one.
- 1 more treat than the previous one, if it is bigger.

By distributing treats like this, we are keeping the invariant that the assignment is optimal for the set of pets that have already received treats. When considering the next pet, we are giving the minimum amount possible. Therefore, we are giving the minimum number of treats whilst also fulfilling the condition.

The sorting of the pets takes $O(N \log N)$ time, and the assigning of the treats can be done in $O(N)$ time. This makes the overall time complexity $O(N \log N)$, which is sufficient to pass Test Set 2.

Analysis: Inconstant Ordering

Test Set 1

It is given that a valid string must start with A.

For $N = 1$, the only block B_1 should therefore start with a character greater than A. To keep building the alphabetically first valid string, we start B_1 with B which is just greater than A. We keep on incrementing the character as we move right to ensure that B_1 is strictly increasing. Notice that we will eventually end up with a prefix of BCDE...XYZ as B_1 . As $L_i \leq 25$, we will never run out of characters for this.

For $N = 2$, we construct the first block B_1 using the process described for $N = 1$. We now evaluate the possible ways to fill the strictly decreasing second block B_2 . We try to start B_2 with the smallest possible character to get the alphabetically first string. This implies that B_2 will end with the smallest character in the alphabet which is A. So, we start filling B_2 from the end with A and move to the left, incrementing the character.

Let b_1 be the last character in B_1 and b_2 be the first character in B_2 . If $b_1 > b_2$, then we will always get a valid string as it ensures that B_2 is strictly decreasing. On the other hand if $b_1 \leq b_2$, then the final string is invalid. For example, if $L_1 = 2$ and $L_2 = 4$, the resulting string from the above process will be ABCDCBA which is invalid.

To ensure that B_2 is always strictly decreasing, we update b_1 with the character just greater than b_2 . This will guarantee that the string is valid and alphabetically the first one. Again as $L_i \leq 25$, we will never run out of characters while filling B_2 . In the above example, ABCDCBA now transforms to a valid string ABEDCBA.

Test Set 2

We generalize the above solution for $N > 2$.

If N is even, we can divide the blocks into $N/2$ pairs as $[L_1, L_2][L_3, L_4] \dots [L_{N-1}, L_N]$ and solve the $N/2$ pairs independently using the same approach as for $N = 2$. We can do this as every such pair of blocks will end with A.

If N is odd, we solve for $N - 1$ blocks using the above approach and for the last block L_N , we treat it as the case when $N = 1$.

The time complexity is $O(L_1 + L_2 + \dots + L_N)$ for each test case.

Analysis: Introductions Organization

Test Set 1

In the first Test Set, the number of people in a new team is so small that we can use brute force. Even in the worst case, we only have five people: three managers and two non-managers. Since an introduction ties up three people, at most one introduction can take place in each minute. Moreover, there are only $\binom{3}{1} \times \binom{4}{2} = 18$ possible groups of an introducing manager plus two other people, and only some of these will meet the conditions needed for an introduction: the introducing manager must know the other two people, and the other two people must not know each other (or else there would be no point in introducing them).

We can use an exhaustive search to explore all ways of performing an introduction each minute until our target people are acquainted, or until we run out of valid introductions to make, and then return the smallest number of introductions we saw in a valid answer (or -1 if there is no answer). The bounds on the number of possible introductions and on the size of the answer (it cannot exceed 3 in this test set) ensure that the space to explore is small, and so our solution will run very quickly.

Test Set 2

To solve Test Set 2 we can model the problem as a [graph](#). Every person in the team is a node, and we connect two different nodes with an edge if the represented people know each other when the team was formed. The key insight to solve this test set is to realize that an introduction changes the state from graph G to a graph H that is G with one extra edge, and the extra edge directly connects two nodes that are connected in G by a [path](#) of length 2 with an intermediate node that is a manager. Let $D_G(v, w)$ be the distance in G between nodes v and w considering just paths that only use managers as intermediate nodes. In what follows, when we say path or distance, we refer to those terms with the restriction of only using managers as intermediate nodes.

Following the reasoning in the previous paragraph, $D_G(v, w) - 1 \geq D_H(v, w)$ — that is, after an introduction, the distance between two nodes decreases by at most 1. Moreover, if $D_G(v, w) > 1$ — that is, v and w do not know each other — and there is a path of managers between them, there is always a way to make an introduction such that the resulting H has $D_H(v, w) = D_G(v, w) - 1$. Let $[v = x_0, x_1, x_2, \dots, x_{D_G(v,w)-1}, x_{D_G(v,w)} = w]$ be a minimum path that between v and w in G . Since v and w do not know each other, x_1 is neither of them, and therefore an intermediate step in the path, so x_1 is a manager. If x_1 introduces $v = x_0$ and x_2 , the path $[v = x_0, x_2, \dots, x_{D_G(v,w)-1}, x_{D_G(v,w)} = w]$ of length $D_G(v, w) - 1$ is formed in the resulting H , so $D_H(v, w) = D_G(v, w) - 1$.

The goal of the problem is to use introductions to reduce the distance between a given pair of nodes v and w to 1 (the minimum possible distance). So, if we could only do one introduction at a time, the answer would be exactly $D_{G_0}(v, w)$ in the initial graph G_0 . Since time considerations don't change whether it is possible for v and w to meet, we can already say that v and w can meet if and only if there is a path connecting them in the initial graph.

Let us now consider simultaneous introductions. Let $G_0, G_1, \dots, G_k$ be a list of graphs where G_i models the state of the team at time i in an optimal way to make v and w meet (meaning k is the answer for this pair). G_0 is the original team and v and w are connected directly by an edge only in G_k . For convenience, let $D_i = D_{G_i}(v, w)$. We can bound D_{i+1} in terms of D_i as follows:

each edge (x, y) on G_{i+1} either exists in G_i or comes from an introduction that happens between minutes i and $i + 1$, meaning there is a path of length 2 $[x, m, y]$ in G_i where m is a manager. Now fix i and consider the path $[v = x_0, x_1, \dots, x_{D_{i+1}-1}, x_{D_{i+1}} = w]$ between v and w in G_{i+1} . Consecutive edges share a person, so at most one of each pair can come from an introduction and the other one must exist in G_i . This gives us a bound $D_i \leq D_{i+1} + \lceil (D_{i+1}/2) \rceil$. Let us show that this bound is tight and always reachable: if we are at state G_i , and $D_i > 1$, we can take a minimum path $[v = y_0, y_1, \dots, y_{D_i-1}, y_{D_i} = w]$ between v and w in G_i and have y_{3j+1} to introduce y_{3j} and y_{3j+2} for each j between 0 and $\lceil ((D_i - 2)/3) \rceil$. Those introductions at minute i yield $D_{i+1} = D_i - \lceil ((D_i - 2)/3) \rceil$ and make exactly $\lceil (D_{i+1}) \rceil$ edges in G_{i+1} come from an introduction.

Given all the theory above, the answer for a given pair v and w for which $D_{G_0}(v, w)$ is undefined (that is, they are not connected) is impossible, and is otherwise $f(D_{G_0}(v, w))$ where $f(x)$ is the number of steps it takes for $x := x - \lceil ((x - 1)/3) \rceil$ to reach $x = 1$. Notice that it takes a logarithmic number of steps to reach 1, so we can just calculate f on demand, or we can memoize all the needed values (for x between 1 and $\mathbf{N}$). Most shortest-path algorithms can be modified slightly to calculate the distance D_0 considering only managers as intermediate nodes as above, but the simplest must be [Floyd-Warshall](#). Floyd-Warshall computes the shortest paths between every pair of nodes by considering each possible intermediate node in its outermost loop. So, simply restricting that loop to iterate over managers (up to $\mathbf{M}$ instead of $\mathbf{M} + \mathbf{N}$) is enough to calculate, in time $O(\mathbf{M} \times (\mathbf{N} + \mathbf{M})^2)$, the distance D_{G_0} for every pair of nodes. After running Floyd-Warshall and memoizing f once, we can get the answer for every pair in constant time, so the algorithm takes $O(\mathbf{M} \times (\mathbf{M} + \mathbf{N})^2 + \mathbf{P})$ time overall.

Analysis: Irrefutable Outcome

Test Set 1

We can determine the outcome by playing through the game. For each turn: if the current player cannot move, the game is over. If the current player can only move on one side, eliminate the outermost piece from that side. Otherwise, the current player must choose between the leftmost piece and the rightmost piece.

Notice that this choice leads to a subgame where it is the other player's turn. (E.g., the starting game $\text{I}\text{O}\text{I}\text{I}$ where Izabella plays first can lead to either OII or IOI where it is Olga's turn.) If there is only one piece left in a subgame, the current player wins if they can take the piece and loses if they cannot.

For each turn, eliminate the piece that the current player takes. If the current player can choose between both sides, solve both subgames and choose the option that is more favorable to the current player. Continue until the game has finished.

If N is the length of the input string $\mathbf{B}$, there are at most N moves in the game, and there are at most 2 options per-move, so this solution requires at most $O(2^N)$ time. Moreover, one player having a choice means the pieces at both ends are of their color. Since only one of them is removed, that means the next player cannot have more than one option. This means that at most half of the moves actually have 2 options, meaning our algorithm's running time can be bounded more tightly by $O(2^{N/2})$.

Test Set 2

The solution above can be optimized: use [memoization/dynamic programming](#) to avoid playing a subgame more than once. For each subgame, save its result and if you were to play it again, simply lookup the result instead of doing the recursion. Since there are less than N^2 subgames, this can yield a time complexity as low as $O(N^2)$. Implementations that manipulate strings, use them to represent the states or use inefficient ways to do the lookups can have higher time complexities, but the bounds of this test set are small enough that we can get away with doing it.

Test Set 3

We can alternatively determine the outcome without subgames. For each turn: if the current player cannot move, the game is over. If the current player can only move on one side, eliminate the outermost piece from that side.

If the current player can move on either side, the end state of the game can be calculated as follows: if the I and O pieces continue to alternate, the last player to take a piece wins (with zero pieces left). Otherwise, the game will end once the players reach a pair of consecutive identical pieces (II or OO).

Suppose the current player (who can move on either side) is Izabella. From each of the sides, find the nearest pair of consecutive identical pieces (II or OO). If one of those pairs is II , Izabella can win by choosing the outermost piece from that side (as Olga will not be able to move after Izabella takes one of the I s in II). Note that if both sides have a II pair, choosing

the side with the closer pair will maximize Izabella's score — which can be calculated as the number of pieces between that II and the outermost piece on the other side, inclusive.

Alternatively, if the nearest pairs on both sides are OO , Izabella will ultimately lose the game. She can minimize Olga's score by eliminating the alternating I and O pieces from both sides until reaching the OO(s) . Then the score will be the number of pieces between the two occurrences of OOs , 1 if there is only one OO or 2 if there are overlapping OOs , that is, OOO , inclusive.

This solution takes linear time.